

ASSIGNMENT-13.4

NAME-K.BHAVYA SRI

H.NO-2303A51863

BATCH-13

Task 1: Refactoring Data Transformation Logic

Scenario: You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output.

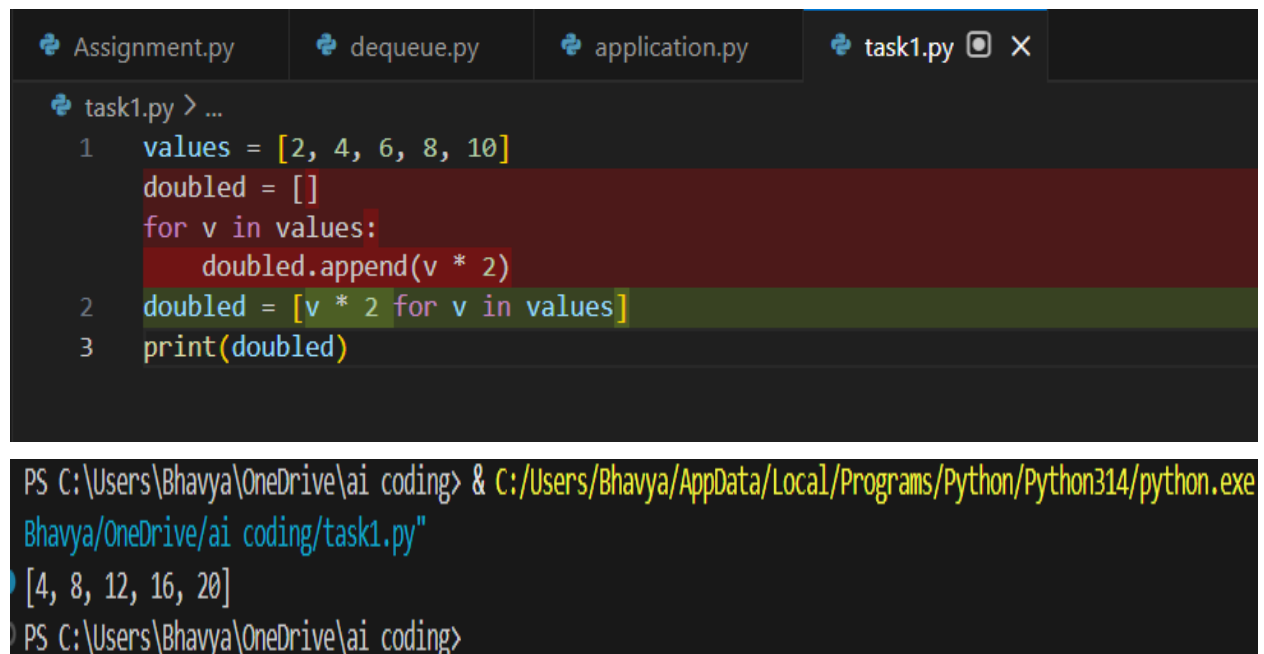
Legacy Code

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Expected Output

```
[4, 8, 12, 16, 20]
```

Before Refactoring Code along with Output:



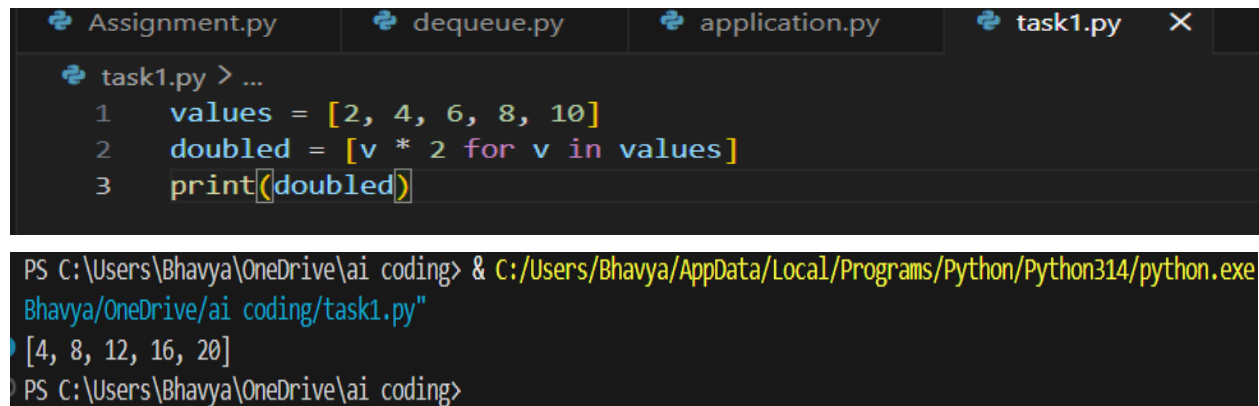
The screenshot shows a code editor with four tabs: Assignment.py, dequeue.py, application.py, and task1.py. The task1.py tab is active, displaying the following Python code:

```
1 values = [2, 4, 6, 8, 10]
   doubled = []
   for v in values:
       doubled.append(v * 2)
2 doubled = [v * 2 for v in values]
3 print(doubled)
```

Below the code editor, a terminal window shows the command to run the script and its output:

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe
Bhavya/OneDrive/ai coding/task1.py"
[4, 8, 12, 16, 20]
PS C:\Users\Bhavya\OneDrive\ai coding>
```

After Refactoring Code along with Output:



```
task1.py > ...
1 values = [2, 4, 6, 8, 10]
2 doubled = [v * 2 for v in values]
3 print(doubled)

PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe
Bhavya/OneDrive/ai coding/task1.py"
[4, 8, 12, 16, 20]
PS C:\Users\Bhavya\OneDrive\ai coding>
```

Justification:

The code has been refactored to use a list comprehension, which is more concise and Pythonic.

The functionality remains the same: it doubles each value in the list and prints the result.

Task 2: Improving Text Processing Code Readability

Scenario: You are working on a log message generator that builds messages word by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

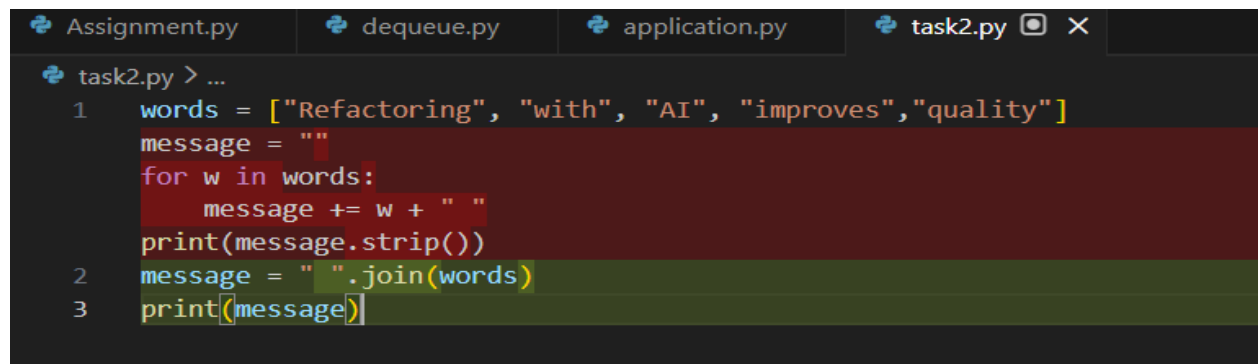
Legacy Code

```
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
```

Expected Output

Refactoring with AI improves quality

Before Refactoring Code along with Output:



```
task2.py > ...
1 words = ["Refactoring", "with", "AI", "improves", "quality"]
  message = ""
  for w in words:
    message += w + " "
  print(message.strip())
2 message = " ".join(words)
3 print(message)
```

```
PS C:\Users\Bhavva\OneDrive\ai coding> & C:/Users/Bhavva/AppData/Local/Programs/Python/Python314/python.exe  
Bhavva/OneDrive/ai coding/task2.py"  
Refactoring with AI improves quality
```

After Refactoring Code along with Output:

```
Assignment.py  dequeue.py  application.py  task2.py  X  
task2.py > ...  
1  words = ["Refactoring", "with", "AI", "improves", "quality"]  
2  message = " ".join(words)  
3  print(message)  
  
PS C:\Users\Bhavva\OneDrive\ai coding> & C:/Users/Bhavva/AppData/Local/Programs/Python/Python314/python.exe  
Bhavva/OneDrive/ai coding/task2.py"  
Refactoring with AI improves quality  
PS C:\Users\Bhavva\OneDrive\ai coding>
```

Justification:

The code has been refactored to use the `join()` method, which is more efficient and Pythonic for concatenating strings from a list. The functionality remains the same: it joins the words with spaces and prints the message.

Task 3: Safer Access to Configuration Data

Scenario: You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
```

```
if "timeout" in config:
```

```
    print(config["timeout"])
```

```
else:
```

```
    print("Default timeout used")
```

Expected Output

Default timeout used

Before Refactoring Code along with Output:

```
Assignment.py  dequeue.py  application.py  task3.py X
task3.py > ...
1  config = {"host": "localhost", "port": 8080}
   if "timeout" in config:
       print(config["timeout"])
   else:
       print("Default timeout used")
2  print(config.get("timeout", "Default timeout used"))

PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe
Bhavya/OneDrive/ai coding/task3.py"
Default timeout used
```

After Refactoring Code along with Output:

```
Assignment.py  dequeue.py  application.py  task3.py X
task3.py > ...
1  config = {"host": "localhost", "port": 8080}
2  print(config.get("timeout", "Default timeout used"))

PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe
Bhavya/OneDrive/ai coding/task3.py"
Default timeout used
```

Justification:

The code has been refactored to use the `get()` method on the dictionary, which is more concise and Pythonic. It achieves the same functionality: if "timeout" is in the config, it prints the value; otherwise, it prints the default message.

Task 4: Refactoring Conditional Logic for Scalability

Scenario: You are enhancing a utility module that performs different operations based on user input.

Task Description

- Examine the multiple if-elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

action = "divide"

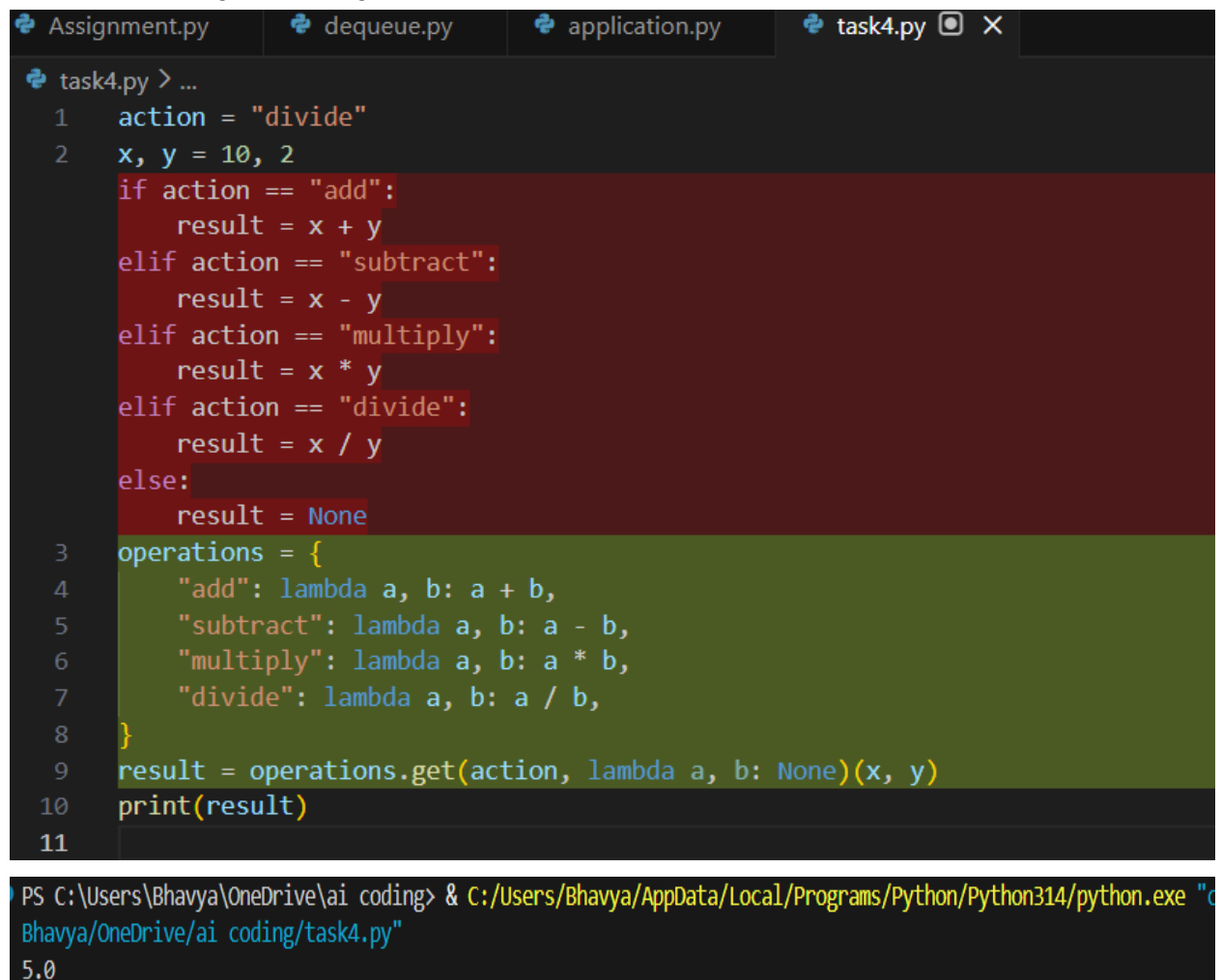
x, y = 10, 2

```
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
```

Expected Output

5.0

Before Refactoring Code along with Output:



The screenshot shows a Python IDE with four tabs: Assignment.py, dequeue.py, application.py, and task4.py. The task4.py tab is active, showing the following code:

```
1  action = "divide"
2  x, y = 10, 2
   if action == "add":
       result = x + y
   elif action == "subtract":
       result = x - y
   elif action == "multiply":
       result = x * y
   elif action == "divide":
       result = x / y
   else:
       result = None
3  operations = {
4      "add": lambda a, b: a + b,
5      "subtract": lambda a, b: a - b,
6      "multiply": lambda a, b: a * b,
7      "divide": lambda a, b: a / b,
8  }
9  result = operations.get(action, lambda a, b: None)(x, y)
10 print(result)
11
```

Below the code editor, the command prompt shows the execution of the script:

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe "C:\Users\Bhavya\OneDrive\ai coding\task4.py"
5.0
```

After Refactoring Code along with Output:

```
task4.py > ...
1  action = "divide"
2  x, y = 10, 2
3  operations = {
4      "add": lambda a, b: a + b,
5      "subtract": lambda a, b: a - b,
6      "multiply": lambda a, b: a * b,
7      "divide": lambda a, b: a / b,
8  }
9  result = operations.get(action, lambda a, b: None)(x, y)
10 print(result)
11
```

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe "C:\Users\Bhavya\OneDrive\ai coding\task4.py"
5.0
```

Justification:

This refactoring uses a dictionary to map actions to lambda functions, making the code more concise and extensible. It achieves the same functionality as the original code.

Task 5: Simplifying Search Logic in Collections

Scenario: You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.
- Use AI assistance to refactor the logic into a more concise and readable form.
- Maintain the same output behavior.

Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
Expected Output
Item Available
```

Before Refactoring Code along with Output:

```
task5.py > ...
1  inventory = ["pen", "notebook", "eraser", "marker"]
   found = False
   for item in inventory:
       if item == "eraser":
           found = True
           break
2  found = "eraser" in inventory
3  print("Item Available" if found else "Item Not Available")
4

PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe
Bhavya/OneDrive/ai coding/task5.py"
Item Available
PS C:\Users\Bhavya\OneDrive\ai coding>
```

After Refactoring Code along with Output:

```
task5.py > ...
1  inventory = ["pen", "notebook", "eraser", "marker"]
2  found = "eraser" in inventory
3  print("Item Available" if found else "Item Not Available")
4

PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe
Bhavya/OneDrive/ai coding/task5.py"
Item Available
PS C:\Users\Bhavya\OneDrive\ai coding>
```

Justification:

This refactoring uses the `in` operator to check if "eraser" exists in the inventory list, which is more Pythonic and efficient than manually looping through the list with a flag variable.